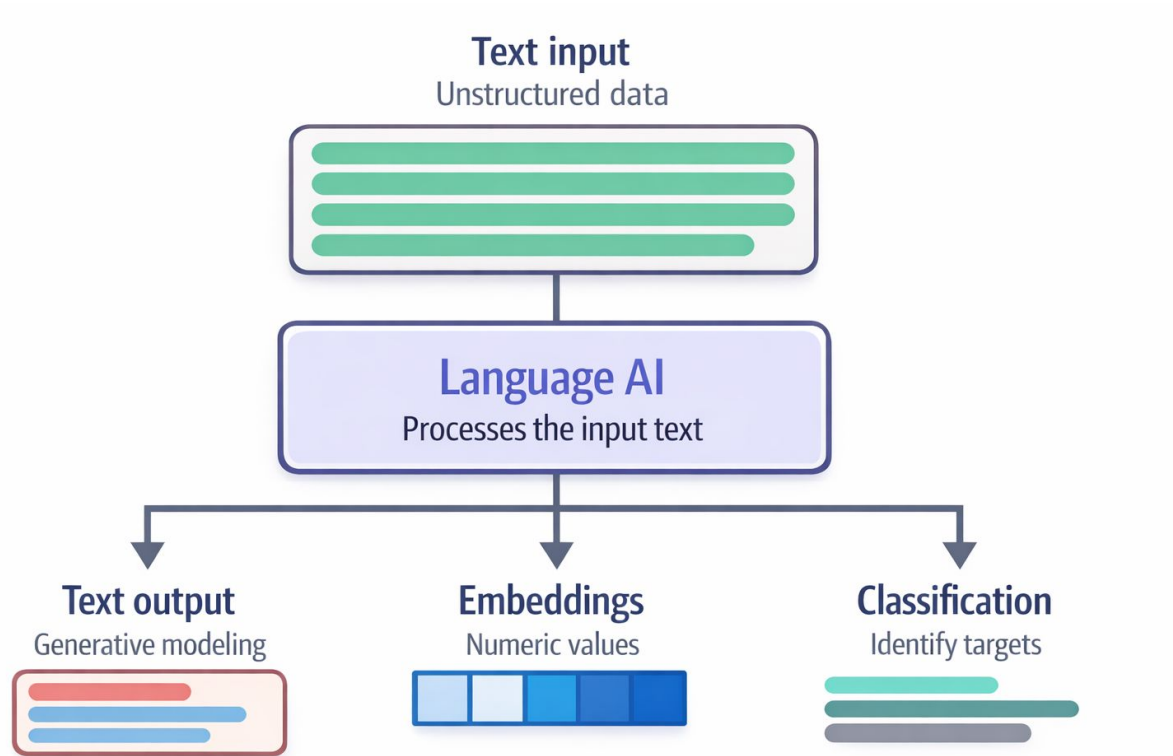


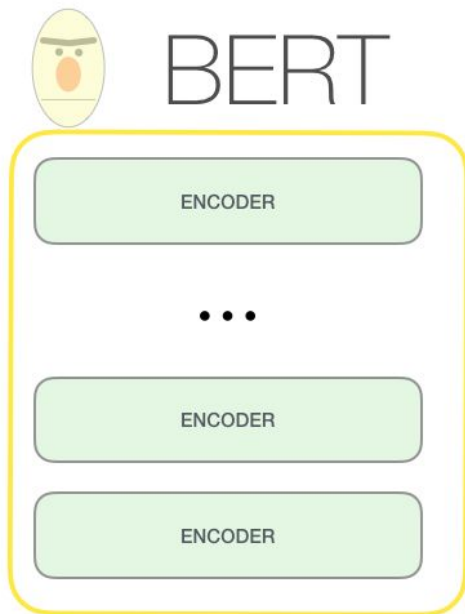
Transformers in Deep Learning

Lecture 4

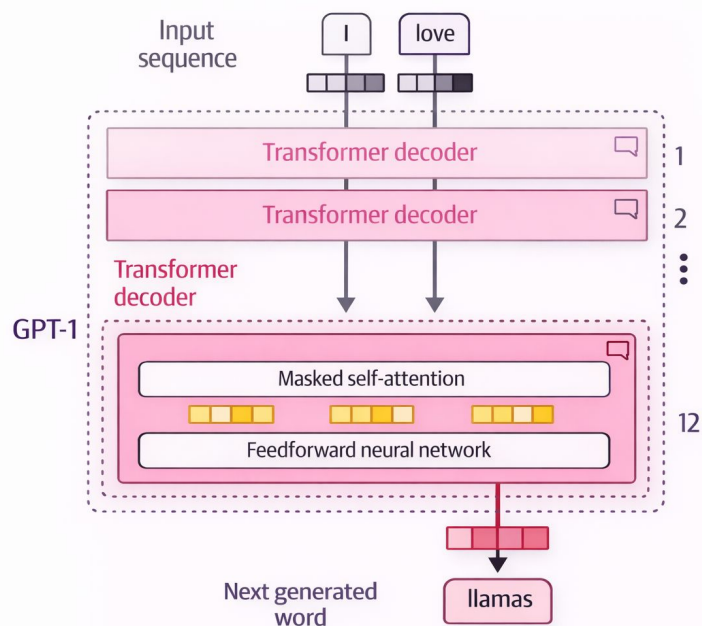
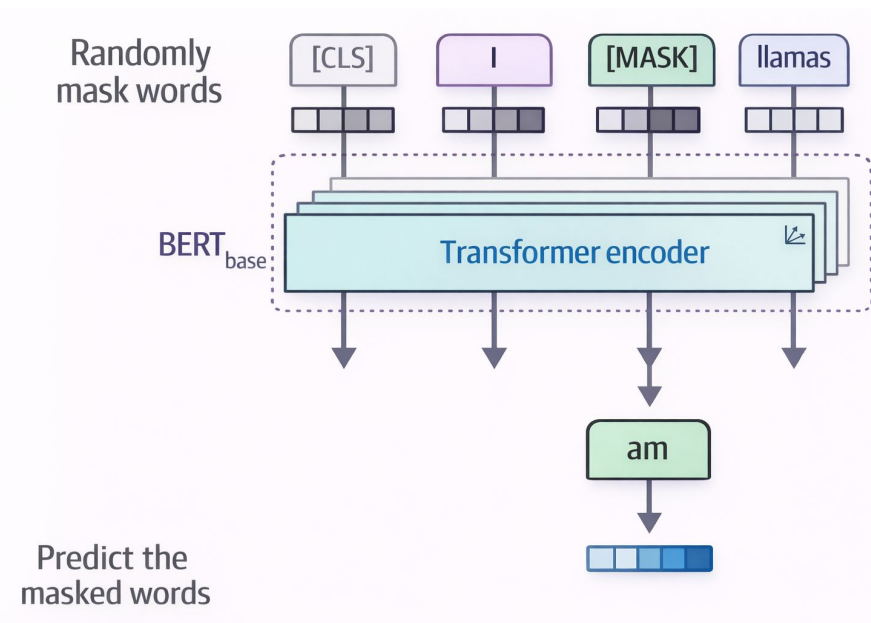
Lecture 3 Recap



Transformers in NLP Journey



BERT and GPT



Computer Vision Tasks

Semantic Segmentation



CAT GRASS
TREE

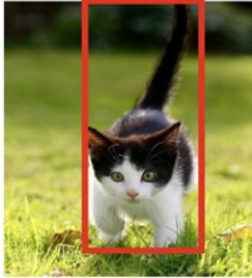
No object
Just pixels

Classification



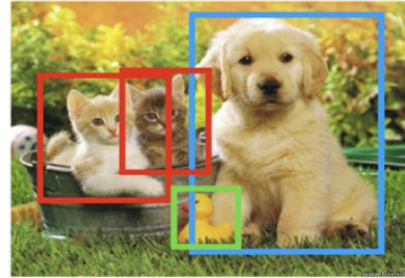
CAT

Classification + localization



CAT

Object detection



CAT DOG DUCK

Instance segmentation

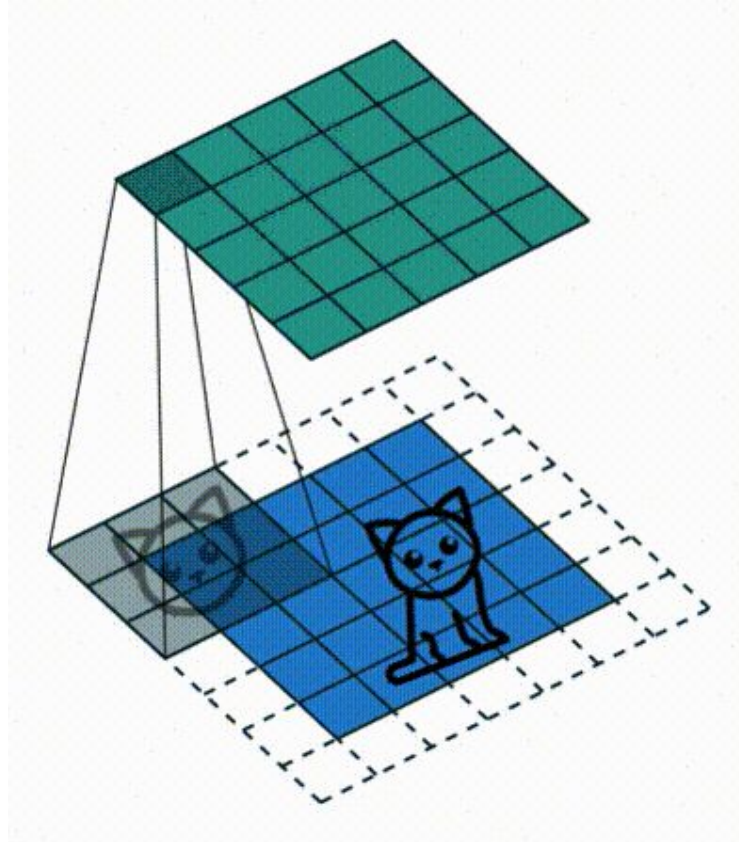


CAT CAT DOG DUCK

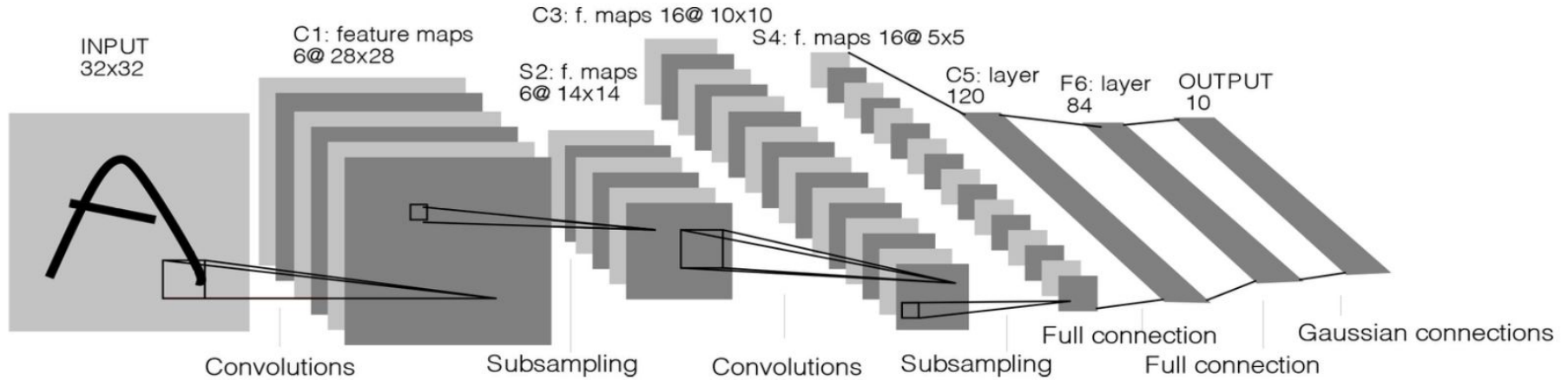
Single object

Multiple objects

Convolutional neural network (CNN)



Computer Vision with CNN



Vision Transformers

Computer vision dominating architecture



Convolutional Neural Networks (CNN)

Transformers

Vision Transformers (ViT)

1989

2017

2020

DINOv2

DeiT

CLIP



AN IMAGE IS WORTH 16X16 WORDS:
TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

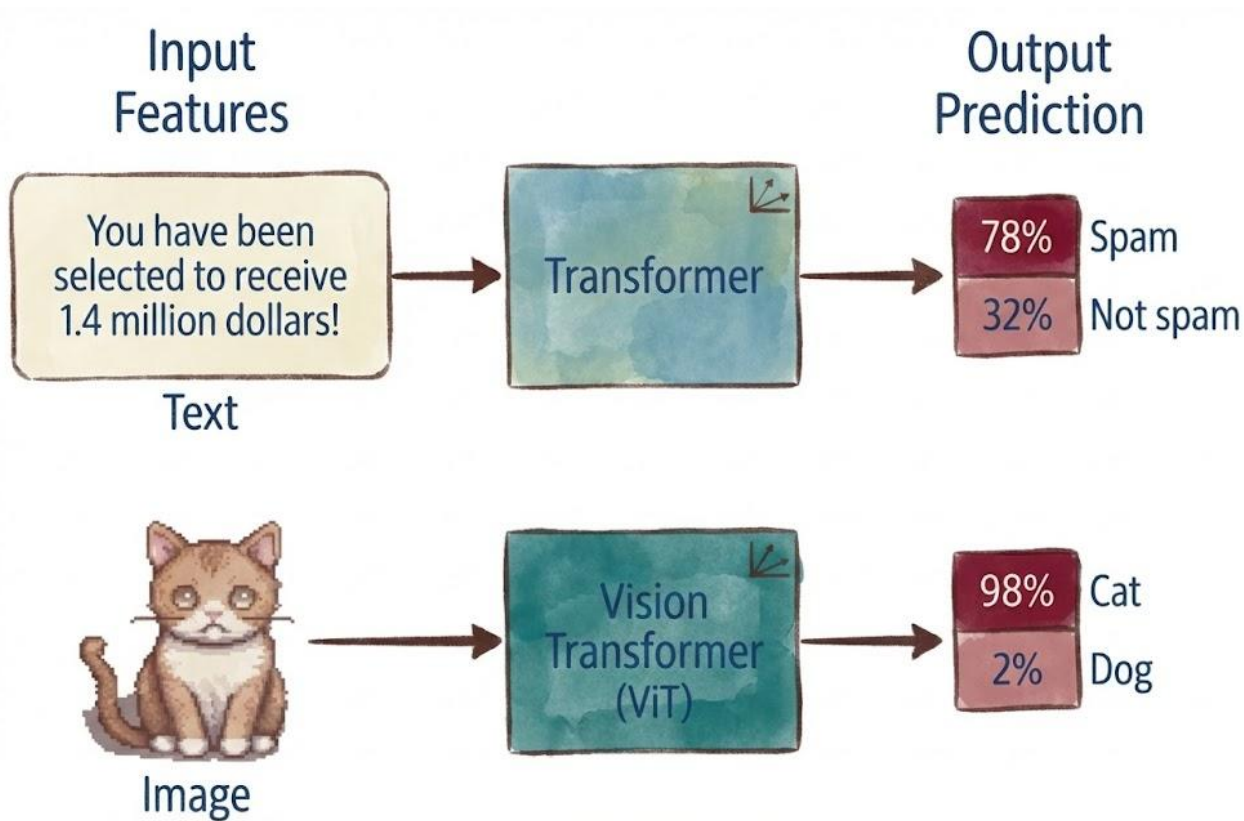
Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}

^{*}equal technical contribution, [†]equal advising

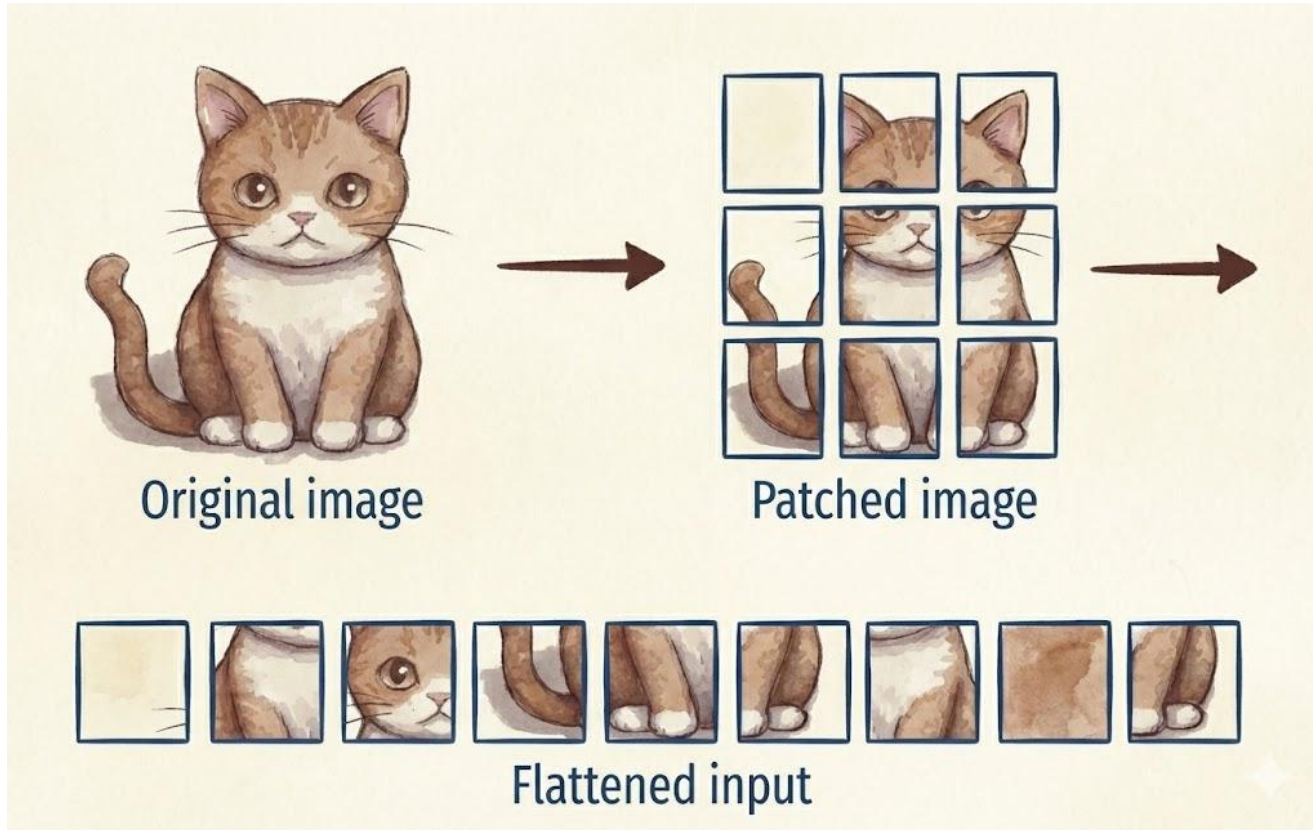
Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

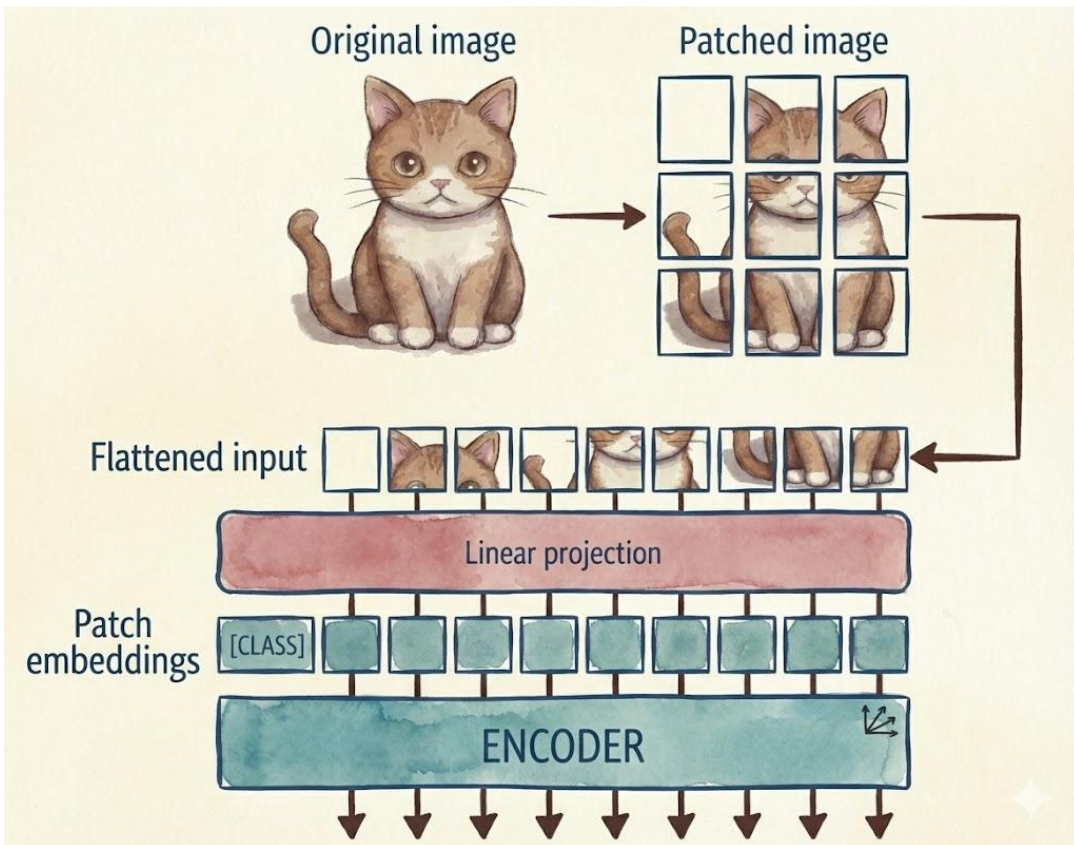
Transformers for Text and Images



Split Image into Patches



Patches == tokens



Vision Transformer (ViT)

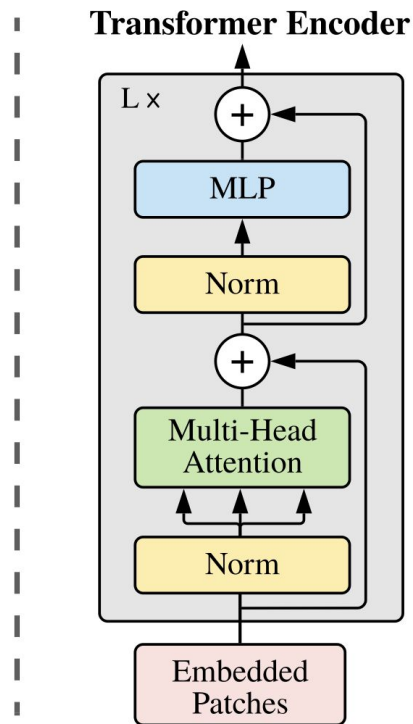
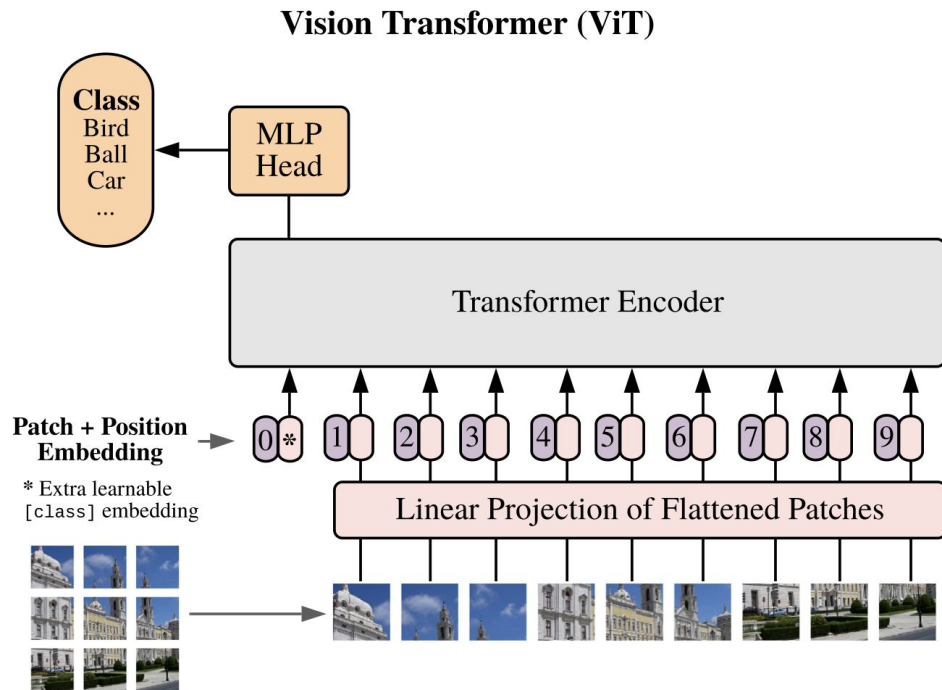
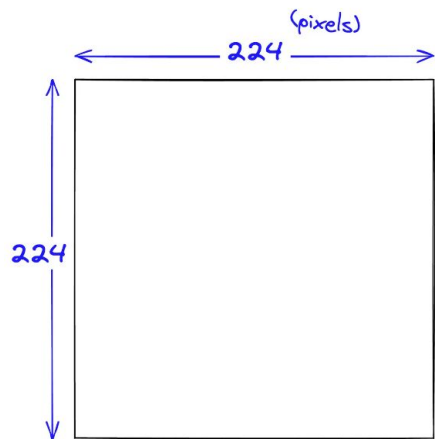


Image Patches



Image

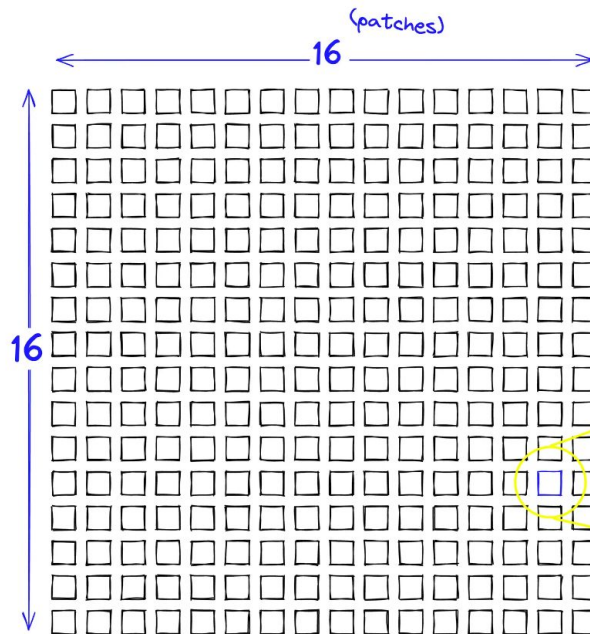
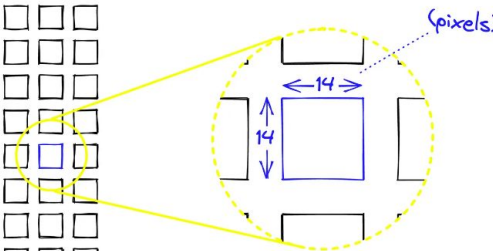
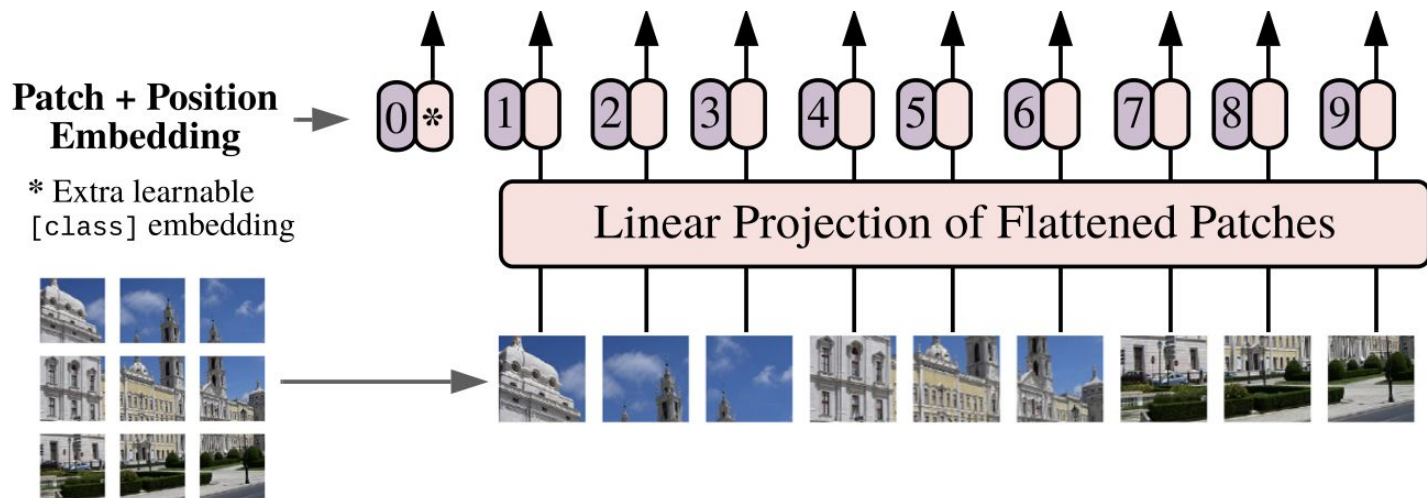


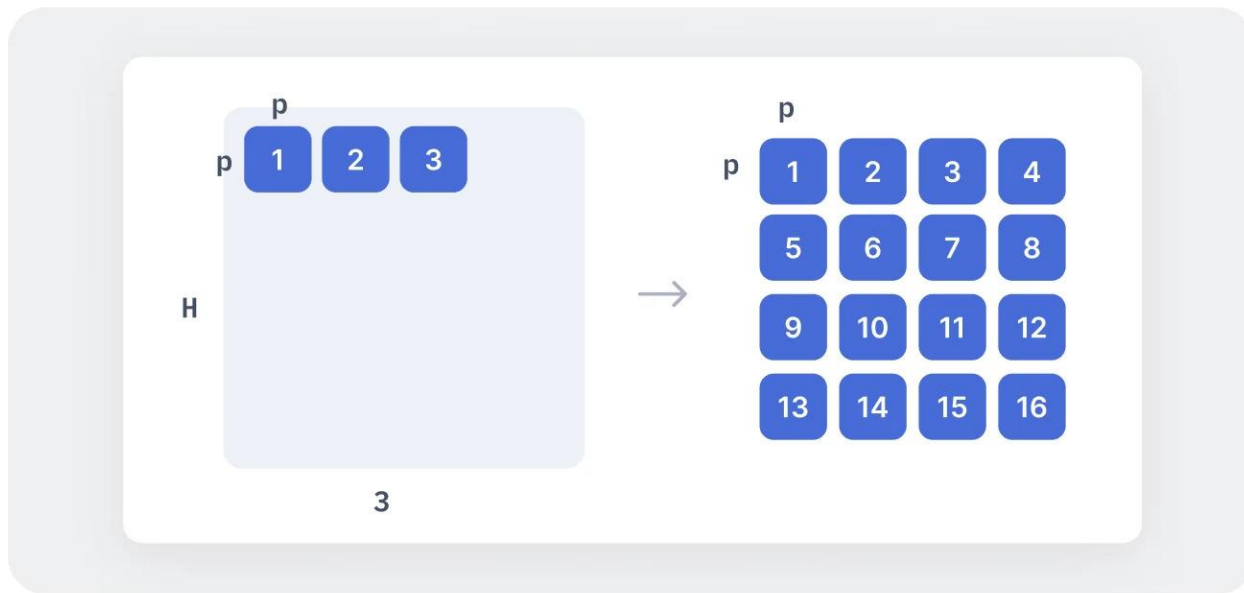
Image Patches



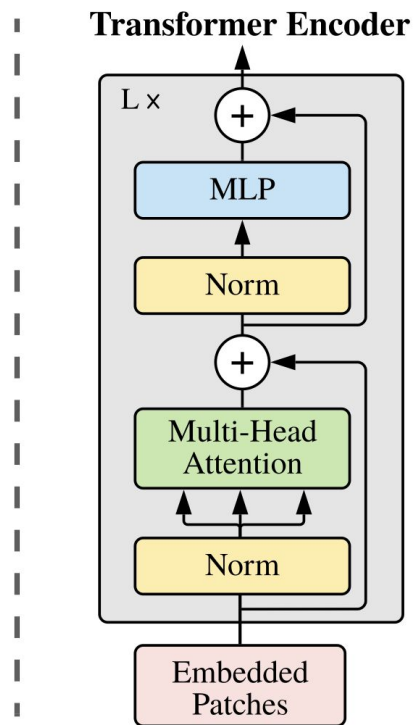
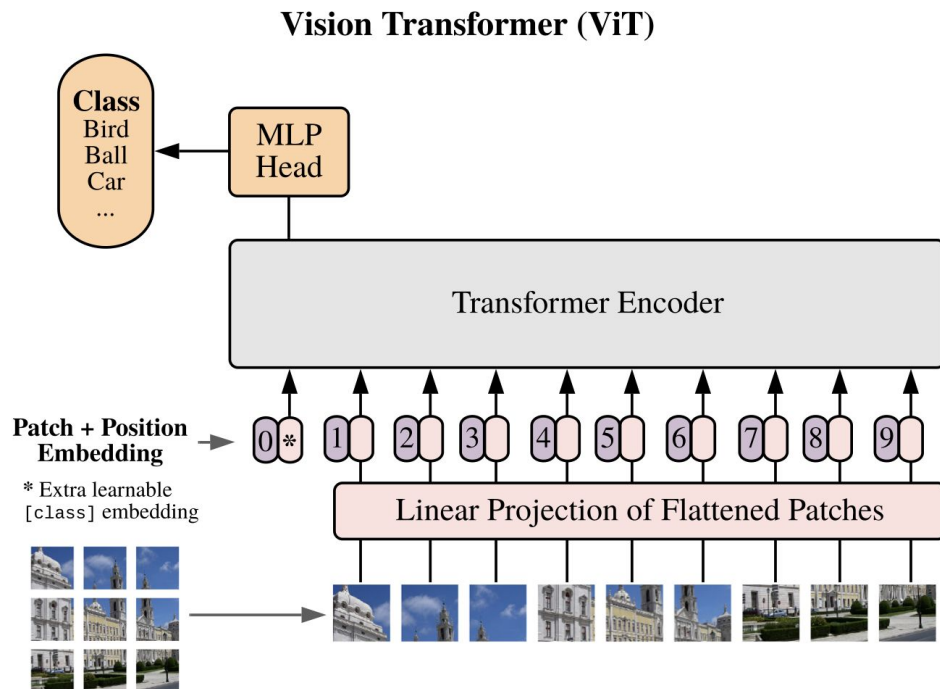
ViT Input Sequence



ViT Positional Encoding



Vision Transformer (ViT)



Multihead Self Attention:

A MULTIHEAD SELF-ATTENTION

Standard **qkv** self-attention (SA, Vaswani et al. (2017)) is a popular building block for neural architectures. For each element in an input sequence $\mathbf{z} \in \mathbb{R}^{N \times D}$, we compute a weighted sum over all values $\mathbf{v}$ in the sequence. The attention weights A_{ij} are based on the pairwise similarity between two elements of the sequence and their respective query $\mathbf{q}^i$ and key $\mathbf{k}^j$ representations.

$$[\mathbf{q}, \mathbf{k}, \mathbf{v}] = \mathbf{z} \mathbf{U}_{qkv} \quad \mathbf{U}_{qkv} \in \mathbb{R}^{D \times 3D_h}, \quad (5)$$

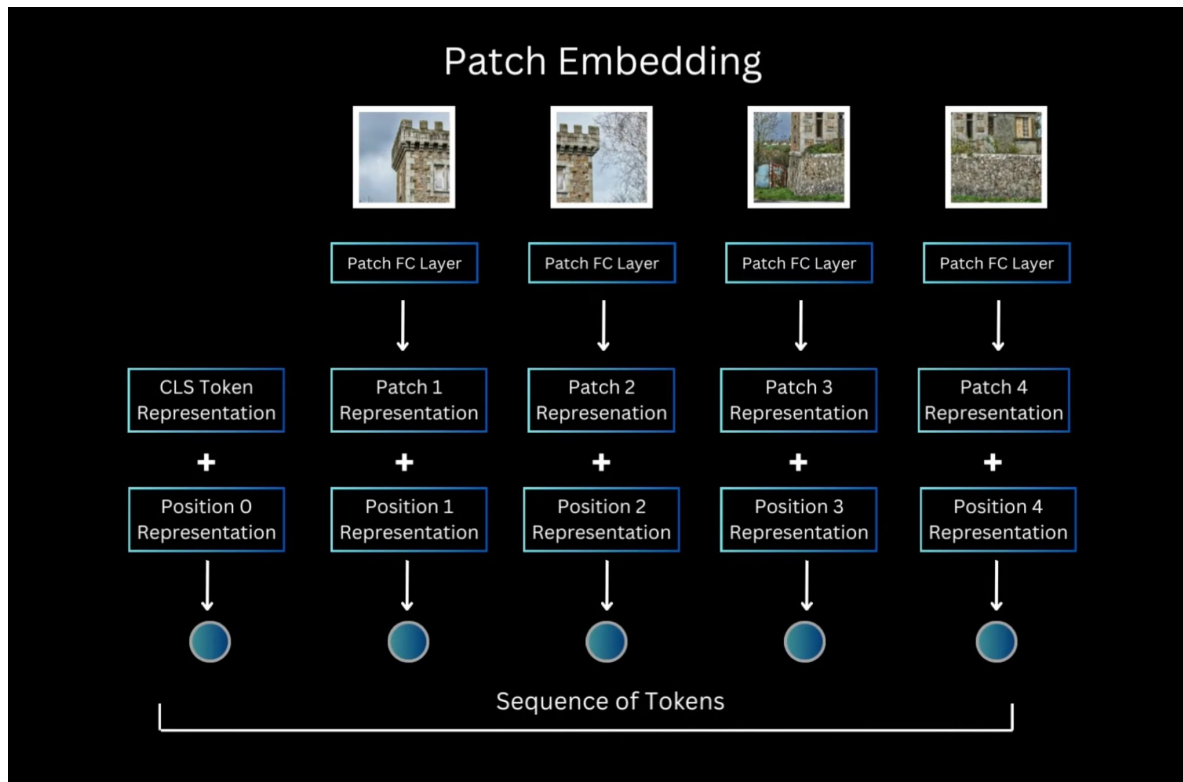
$$A = \text{softmax} \left(\mathbf{q} \mathbf{k}^\top / \sqrt{D_h} \right) \quad A \in \mathbb{R}^{N \times N}, \quad (6)$$

$$\text{SA}(\mathbf{z}) = A \mathbf{v}. \quad (7)$$

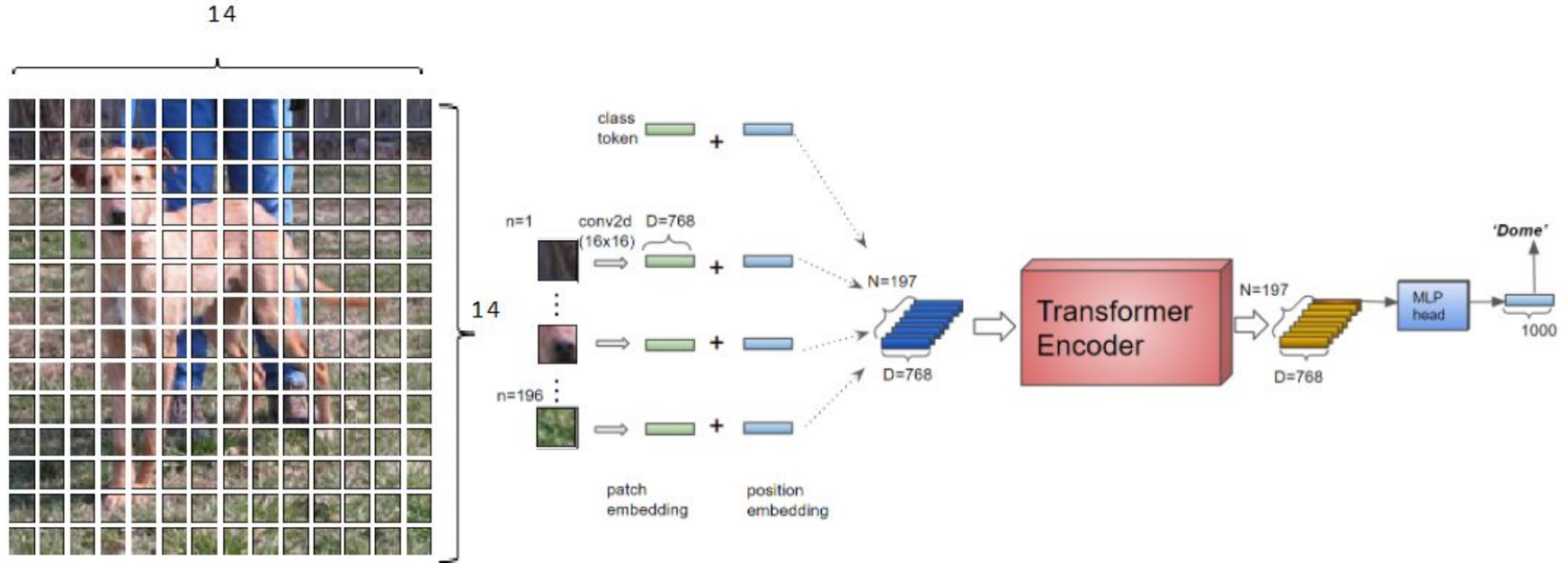
Multihead self-attention (MSA) is an extension of SA in which we run k self-attention operations, called “heads”, in parallel, and project their concatenated outputs. To keep compute and number of parameters constant when changing k , D_h (Eq. 5) is typically set to D/k .

$$\text{MSA}(\mathbf{z}) = [\text{SA}_1(z); \text{SA}_2(z); \cdots; \text{SA}_k(z)] \mathbf{U}_{msa} \quad \mathbf{U}_{msa} \in \mathbb{R}^{k \cdot D_h \times D} \quad (8)$$

How can we get patch embeddings?

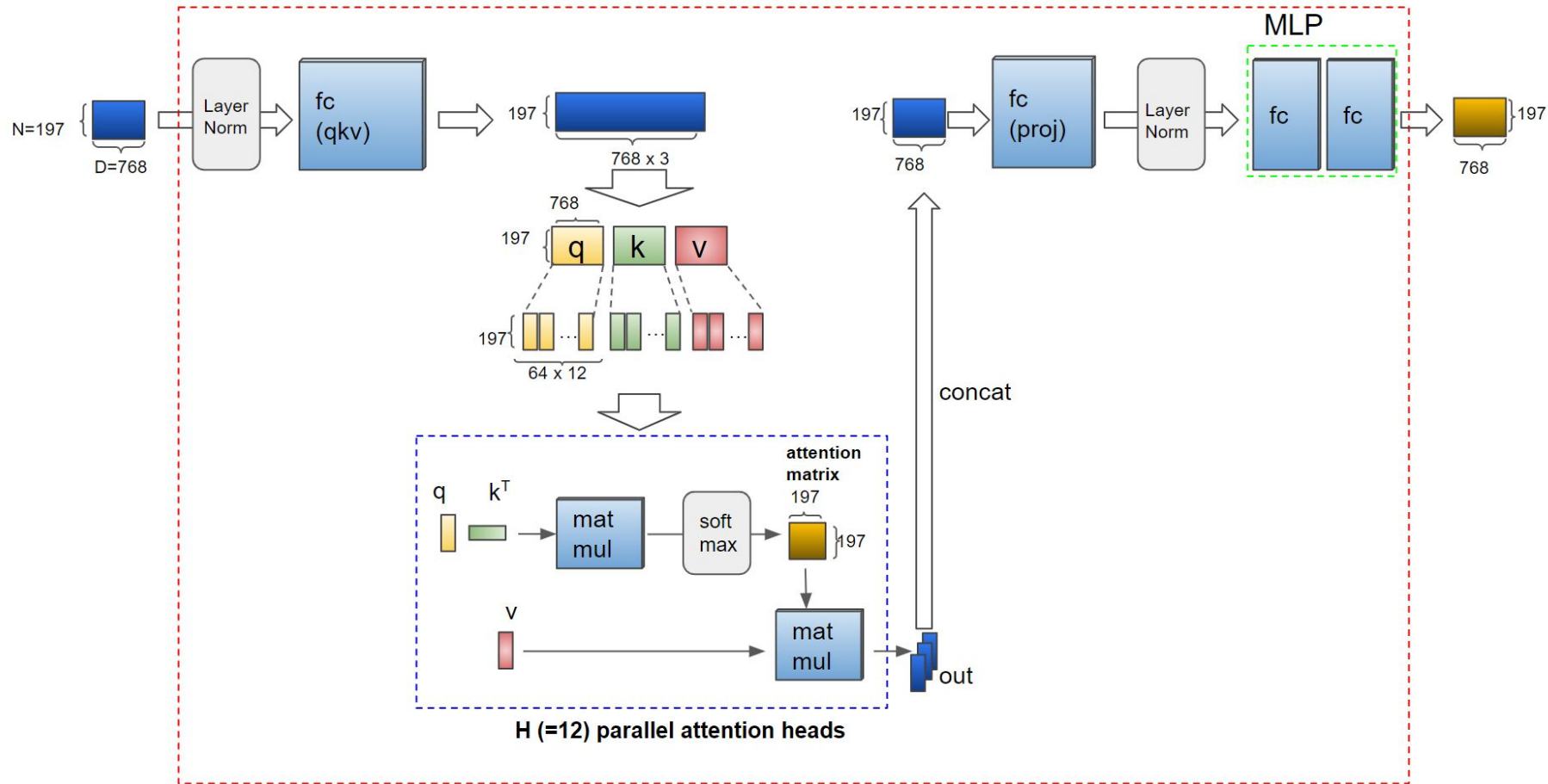


How can we get patch embeddings?



Hybrid Architecture. As an alternative to raw image patches, the input sequence can be formed from feature maps of a CNN (LeCun et al., 1989). In this hybrid model, the patch embedding projection E (Eq. 1) is applied to patches extracted from a CNN feature map. As a special case,

L (=12) series Transformer Encoders



How to pretrain ViT?

- **Supervised:** Use CLS token for classification
- **Self-Supervised:** MASKED patch prediction

We employ the *masked patch prediction* objective for preliminary self-supervision experiments. To do so we corrupt 50% of patch embeddings by either replacing their embeddings with a learnable [mask] embedding (80%), a random other patch embedding (10%) or just keeping them as is (10%). This setup is very similar to the one used for language by Devlin et al. (2019). Finally, we predict the 3-bit, mean color (i.e., 512 colors in total) of every corrupted patch using their respective patch representations.

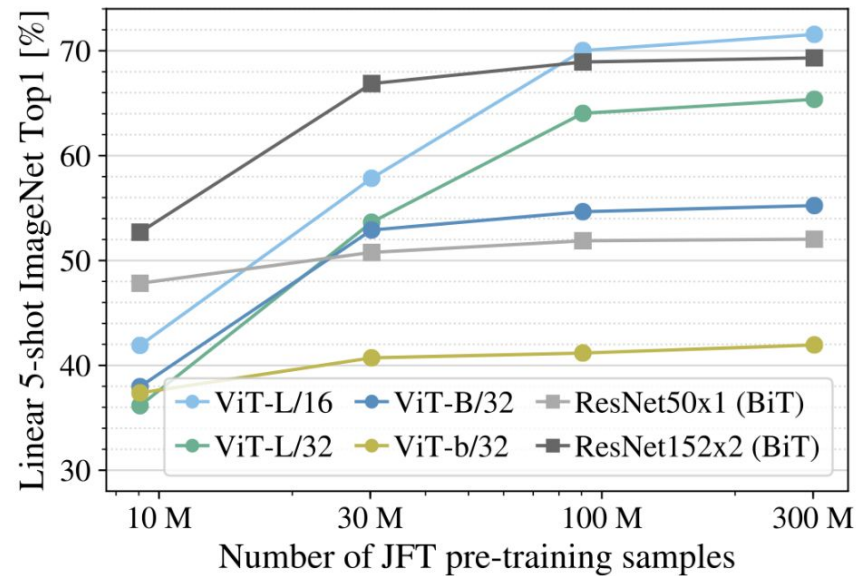
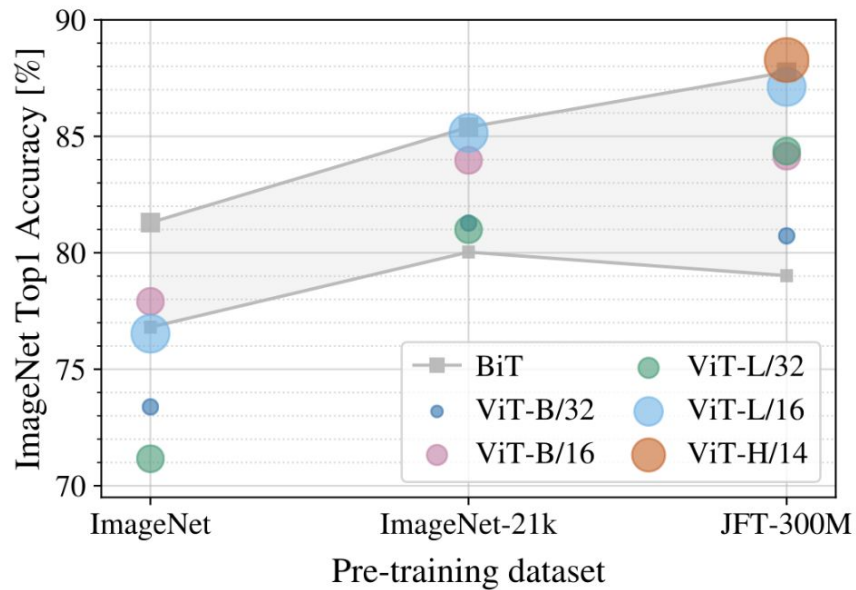
ViT variants

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Table 1: Details of Vision Transformer model variants.

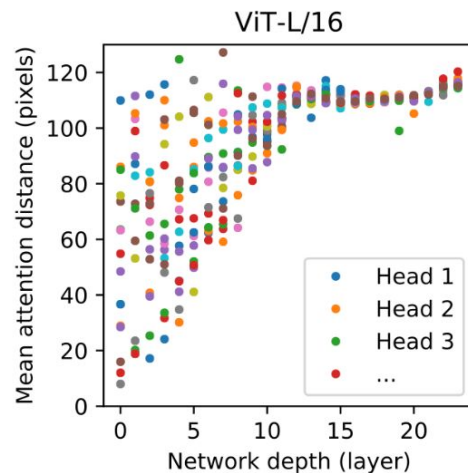
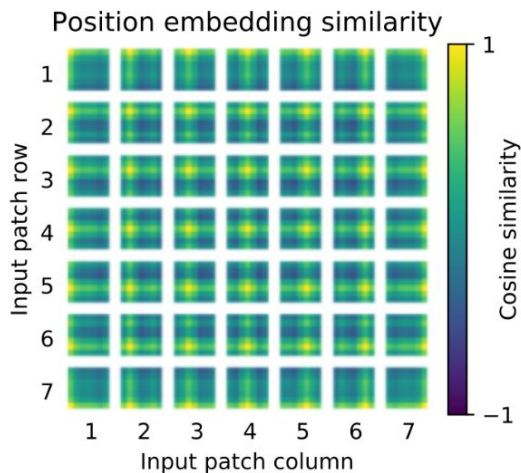
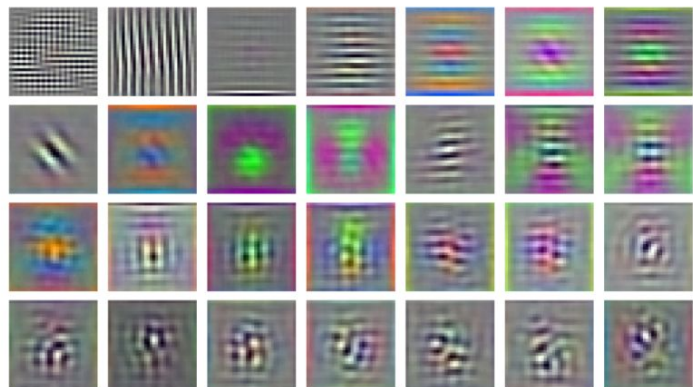
- We need more data for Transformers compared to CNNs
- But Transformers scales better
- **Key idea:** scale model size and data to boost performance

ViT vs CNN scaling

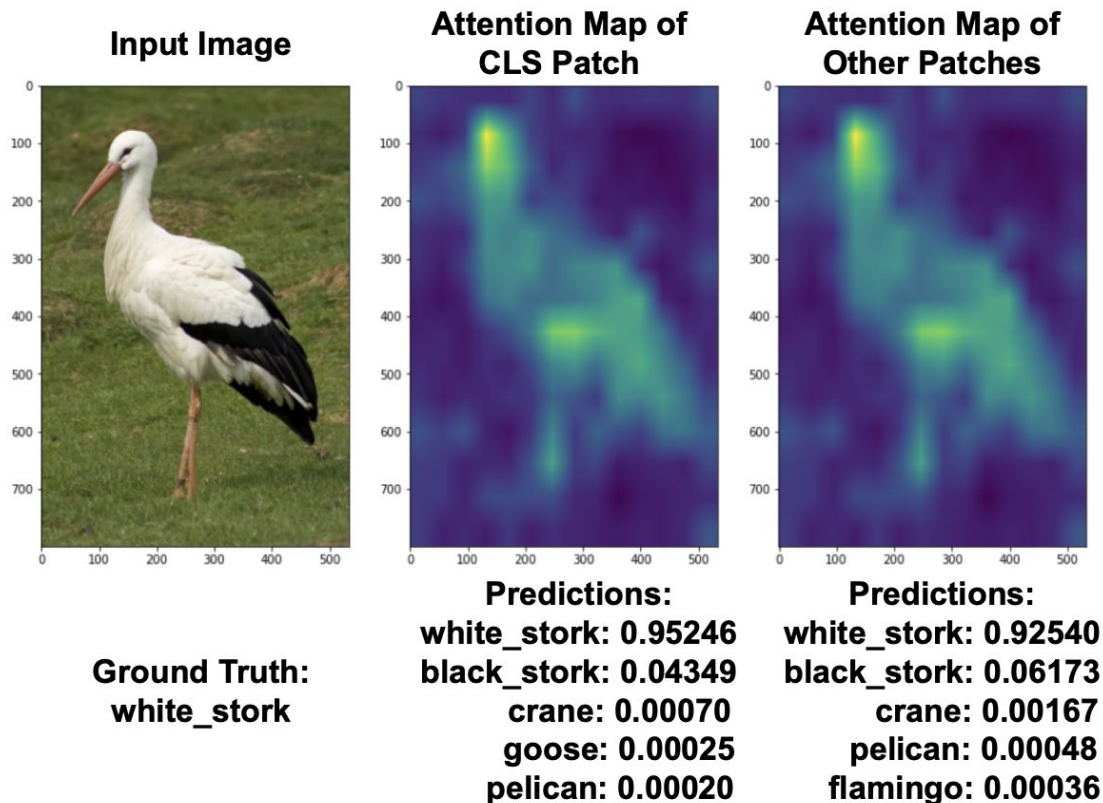


ViT interpretation

RGB embedding filters
(first 28 principal components)



ViT Attention maps



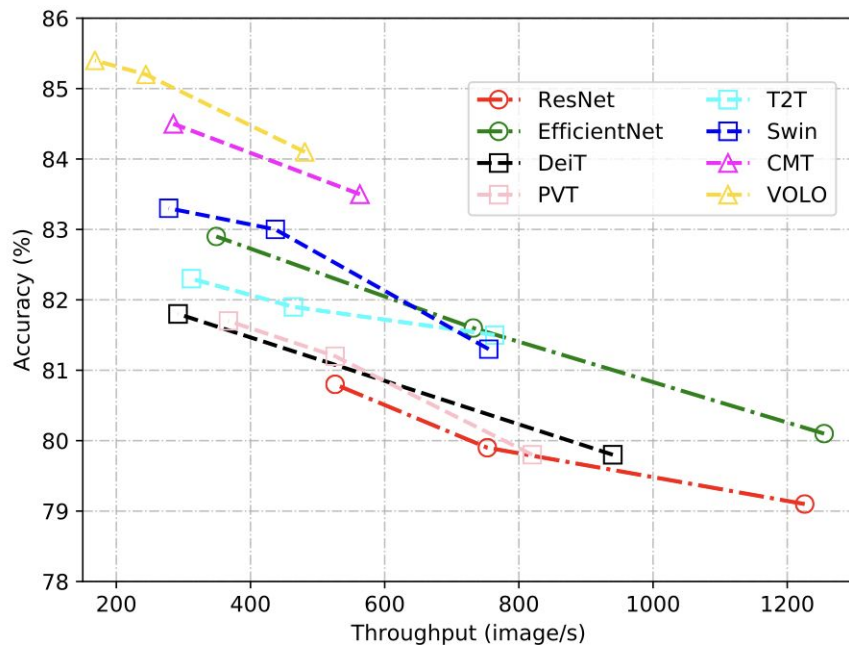
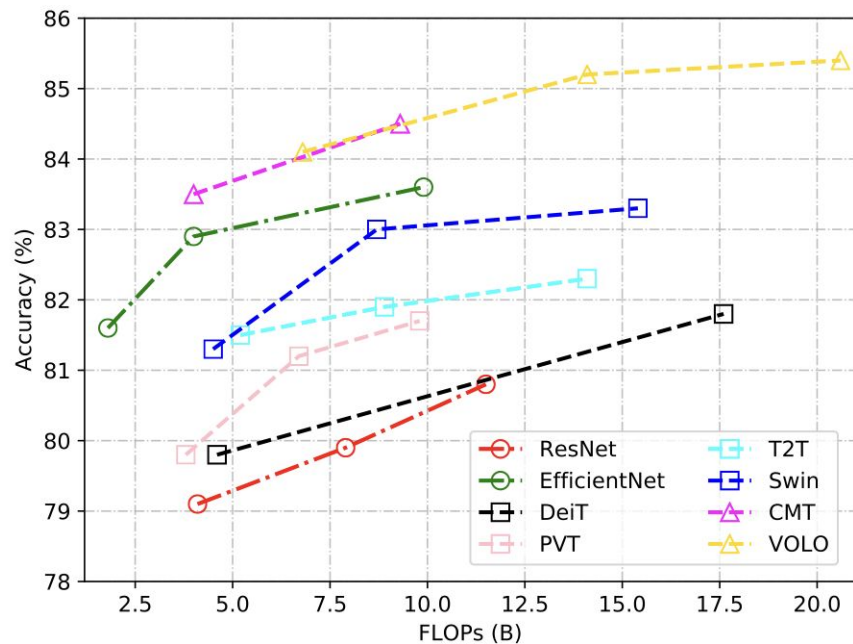
ViT efficiency

Model	Params (M)	FLOPs (B)	Throughput (image/s)	Top-1 (%)
CNN				
ResNet-50 [12], [68]	25.6	4.1	1226	79.1
ResNet-101 [12], [68]	44.7	7.9	753	79.9
ResNet-152 [12], [68]	60.2	11.5	526	80.8
EfficientNet-B0 [94]	5.3	0.39	2694	77.1
EfficientNet-B1 [94]	7.8	0.70	1662	79.1
EfficientNet-B2 [94]	9.2	1.0	1255	80.1
EfficientNet-B3 [94]	12	1.8	732	81.6
EfficientNet-B4 [94]	19	4.2	349	82.9
Pure Transformer				
DeiT-Ti [15], [60]	5	1.3	2536	72.2
DeiT-S [15], [60]	22	4.6	940	79.8
DeiT-B [15], [60]	86	17.6	292	81.8
T2T-ViT-14 [68]	21.5	5.2	764	81.5
T2T-ViT-19 [68]	39.2	8.9	464	81.9
T2T-ViT-24 [68]	64.1	14.1	312	82.3
PVT-Small [73]	24.5	3.8	820	79.8
PVT-Medium [73]	44.2	6.7	526	81.2
PVT-Large [73]	61.4	9.8	367	81.7
TNT-S [29]	23.8	5.2	428	81.5
TNT-B [29]	65.6	14.1	246	82.9
CPVT-S [86]	23	4.6	930	80.5
CPVT-B [86]	88	17.6	285	82.3
Swin-T [61]	29	4.5	755	81.3
Swin-S [61]	50	8.7	437	83.0
Swin-B [61]	88	15.4	278	83.3

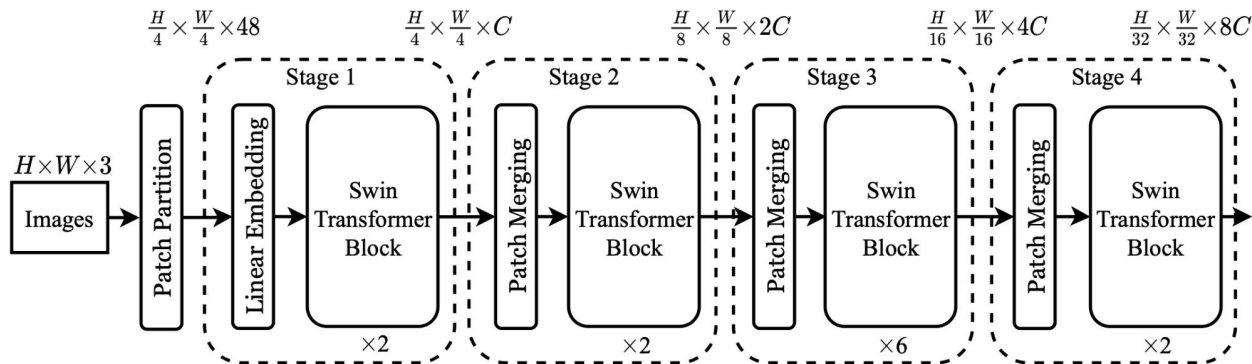
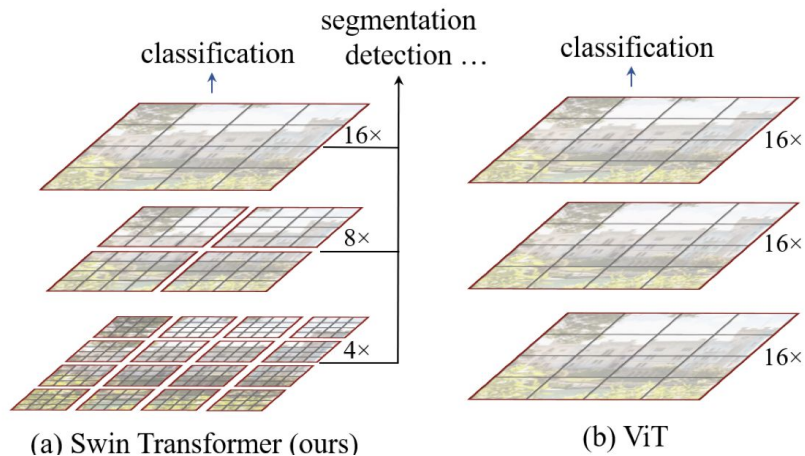
Model	Params (M)	FLOPs (B)	Throughput (image/s)	Top-1 (%)
CNN + Transformer				
Twins-SVT-S [63]	24	2.9	1059	81.7
Twins-SVT-B [63]	56	8.6	469	83.2
Twins-SVT-L [63]	99.2	15.1	288	83.7
Shuffle-T [66]	29	4.6	791	82.5
Shuffle-S [66]	50	8.9	450	83.5
Shuffle-B [66]	88	15.6	279	84.0
CMT-S [95]	25.1	4.0	563	83.5
CMT-B [95]	45.7	9.3	285	84.5
VOLO-D1 [96]	27	6.8	481	84.2
VOLO-D2 [96]	59	14.1	244	85.2
VOLO-D3 [96]	86	20.6	168	85.4
VOLO-D4 [96]	193	43.8	100	85.7
VOLO-D5 [96]	296	69.0	64	86.1

$$\text{FLOPS} = \frac{\text{Number of Floating-Point Operations}}{\text{Execution Time (in seconds)}}$$

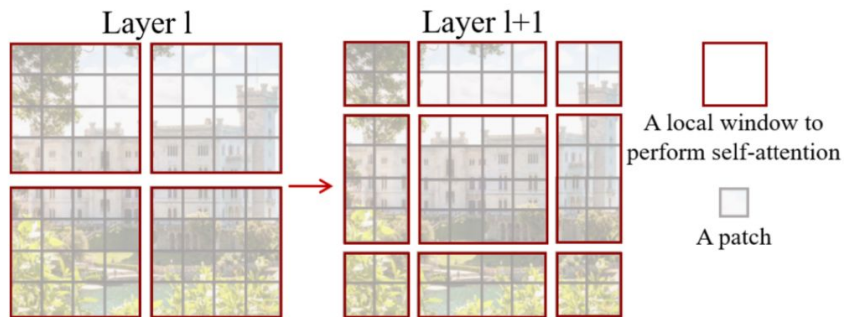
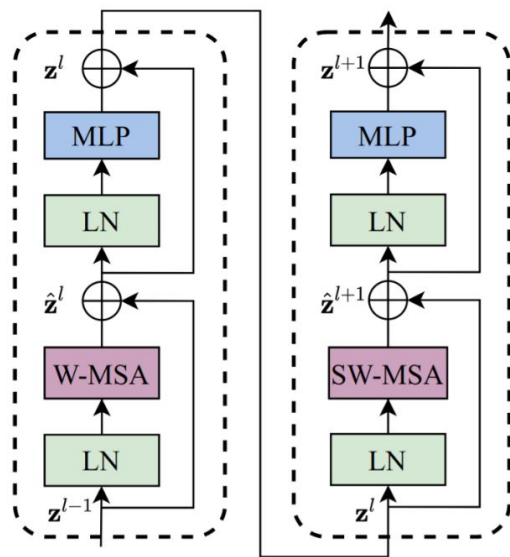
ViT efficiency



Swin Transformer



Shifted Window Attention



Include positional information in self-attention:

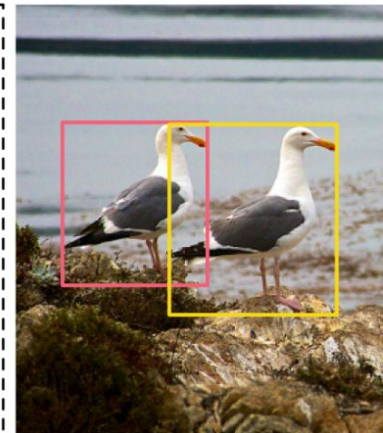
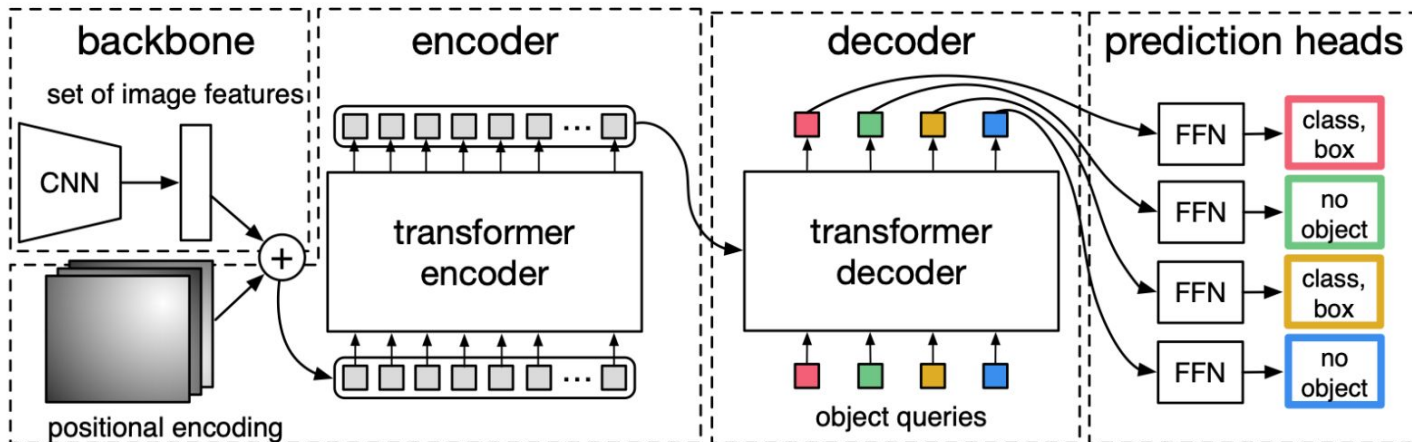
$$SA(q, k, v) = \text{softmax}(qk^t / \sqrt{D_h} + b)v$$

$$b \in \mathbb{R}^{N \times N}$$

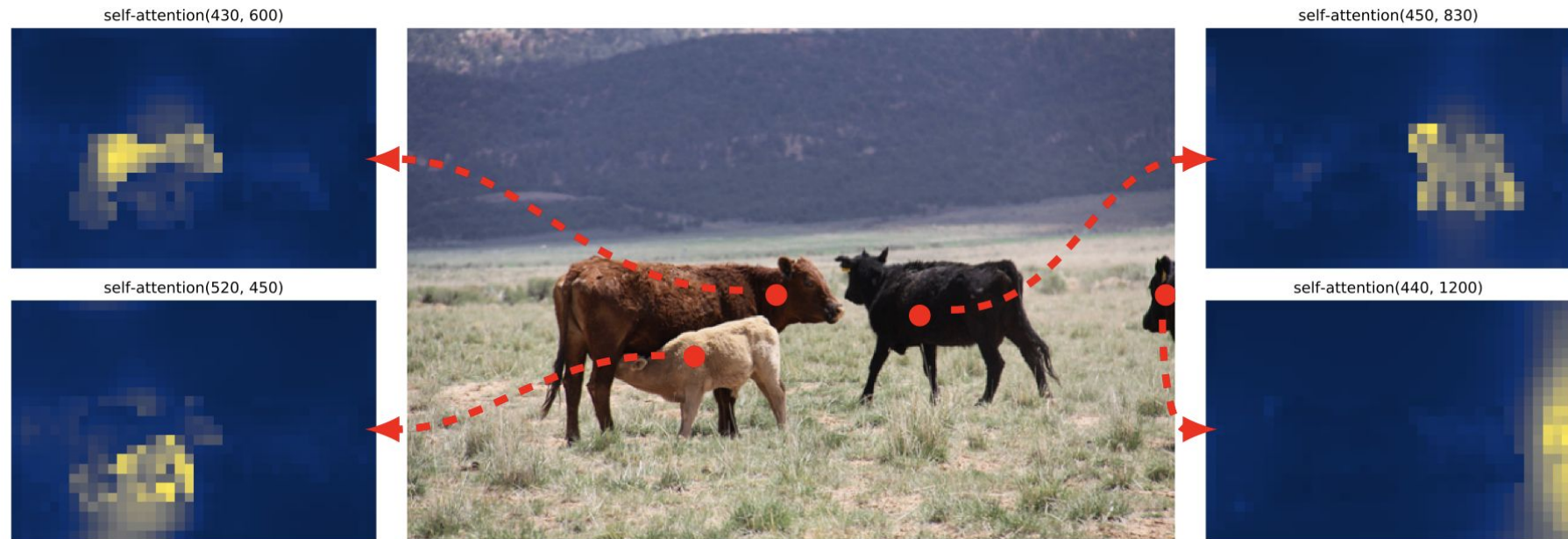
Shifted Window Attention

(a) Regular ImageNet-1K trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [48]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [48]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [48]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [58]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [58]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [58]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [58]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [58]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [63]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [63]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [63]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.5
Swin-B	384 ²	88M	47.0G	84.7	84.5

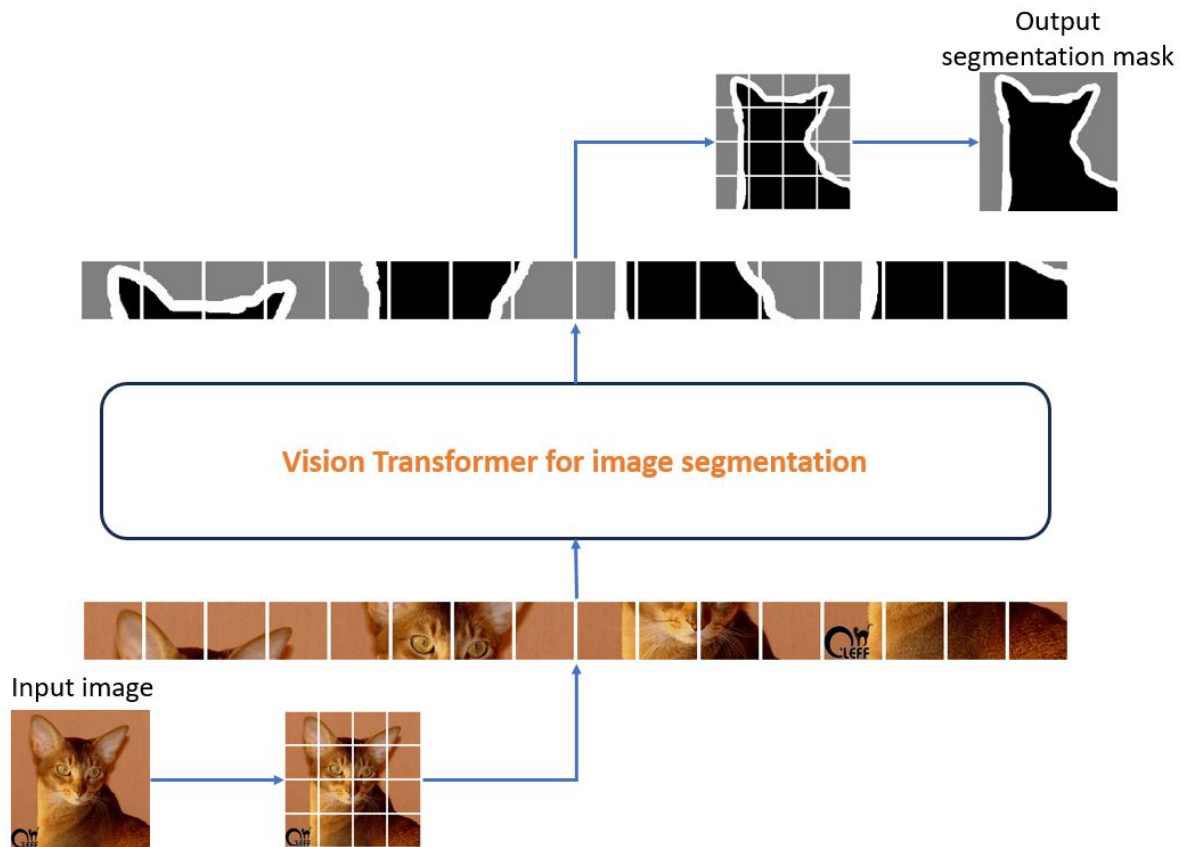
DEtection TRansforme (DETR)



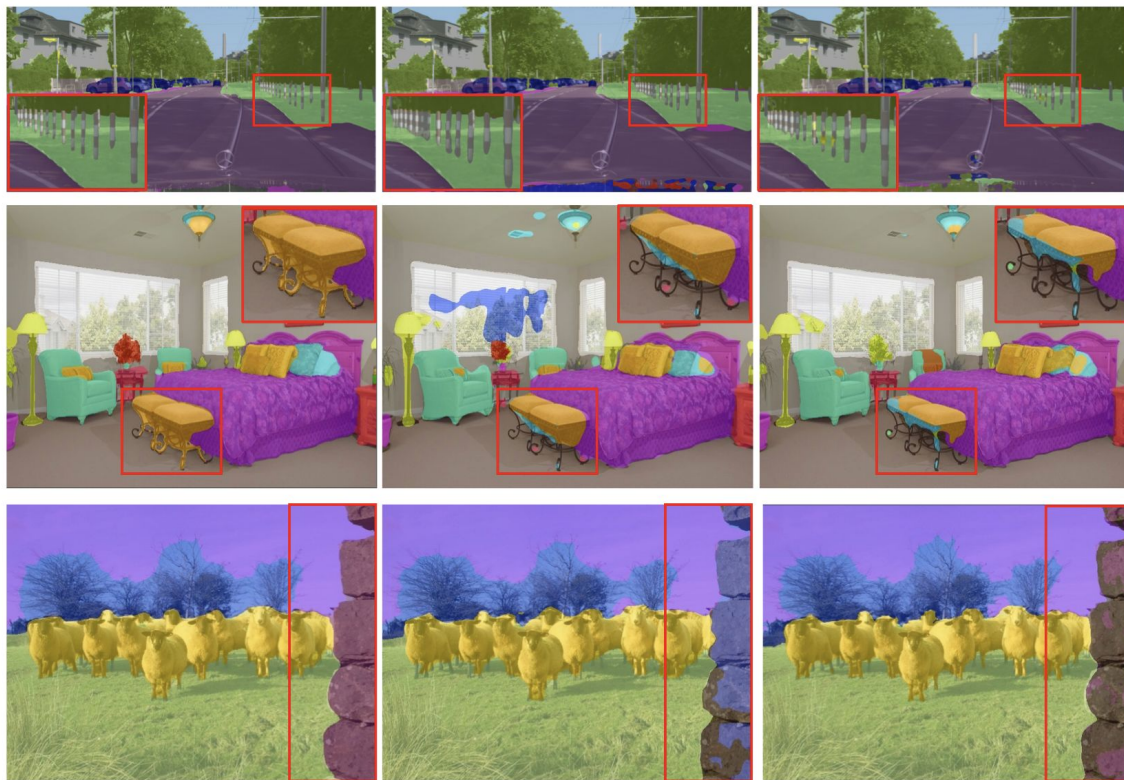
DETR Encoder Attention maps:



Segformer:



Segformer:



SegFormer

SETR

DeepLabV3+

One hammer for everything:

**Semantic
Segmentation**



CAT GRASS
TREE

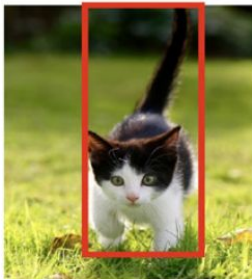
No object
Just pixels

Classification



CAT

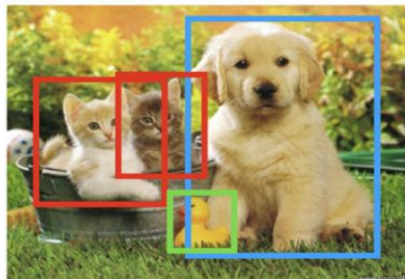
**Classification
+ localization**



CAT

Single object

Object detection



CAT DOG DUCK

Multiple objects

**Instance
segmentation**

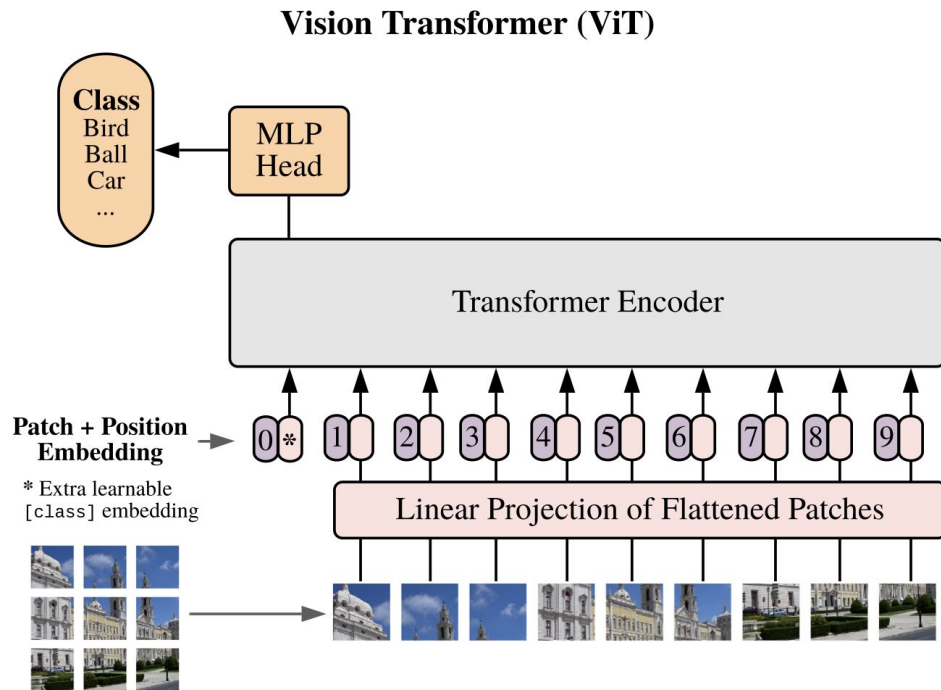


CAT CAT DOG DUCK

One hammer for everything:



One hammer for everything:

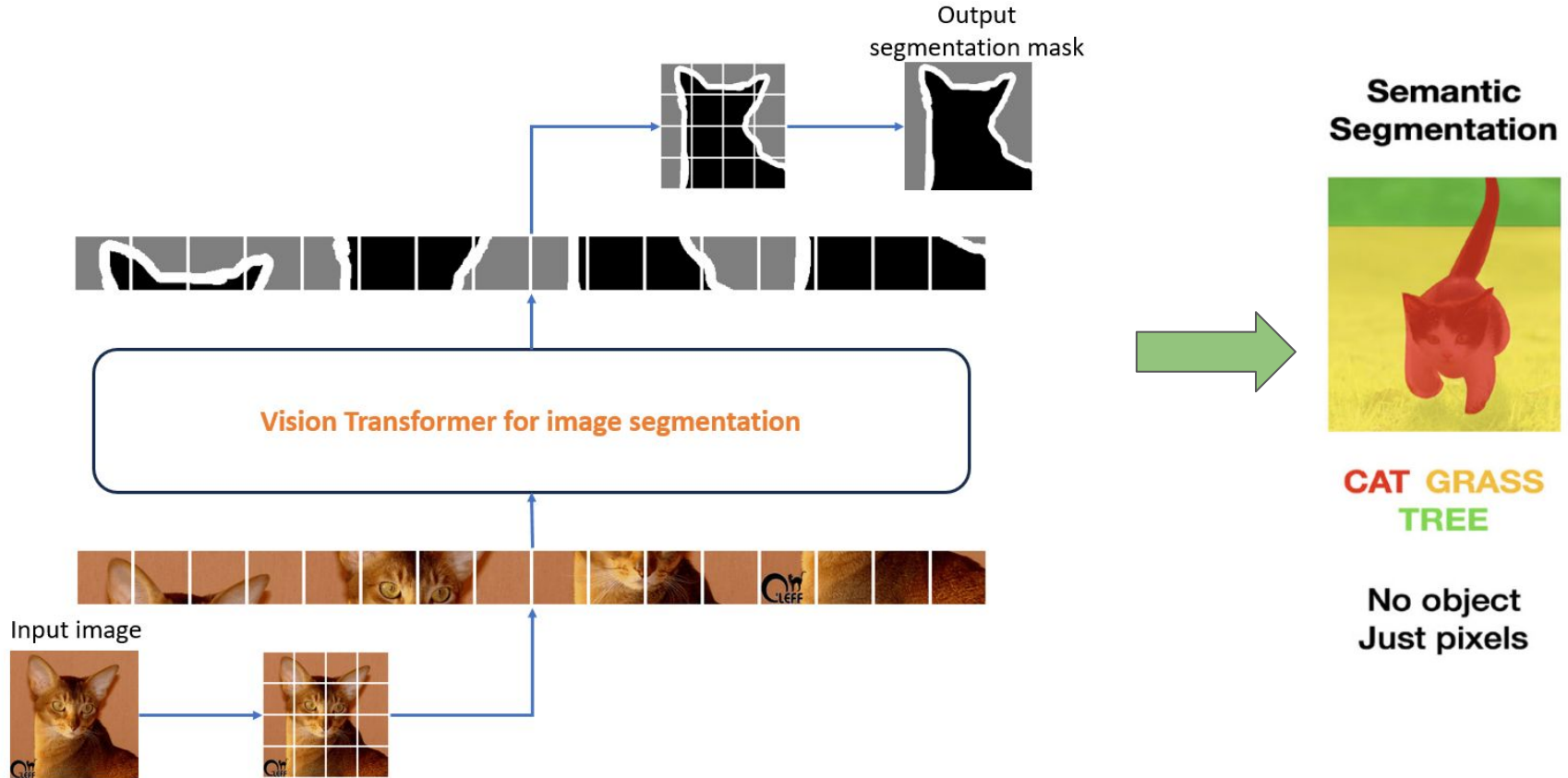


Classification



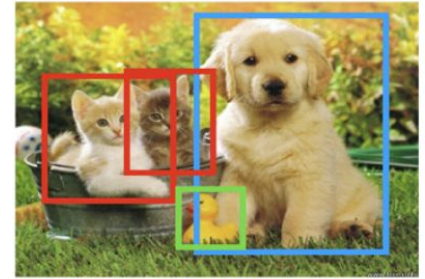
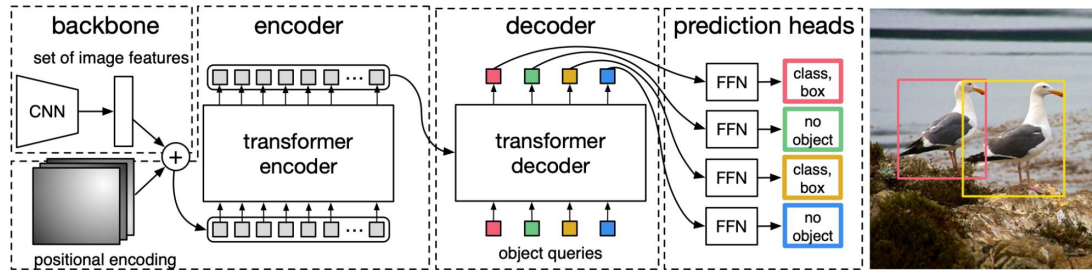
CAT

One hammer for everything:



One hammer for everything:

Object detection



CAT DOG DUCK